In [76]:
```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import regularizers
from tensorflow.keras import layers,models
from tensorflow.keras.layers import Dense,Dropout,Flatten
from tensorflow.keras.initializers import GlorotNormal,HeNormal
```

In [77]:
```
(x_train,y_train),(x_test,y_test)=tf.keras.datasets.cifar10.load_data()
```

In [78]:
```
x_train,x_test=x_train/255.0,x_test/255.0
```

In [79]:
```
#base model
base_model=models.Sequential()
base_model.add(layers.Flatten(input_shape=(32,32,3)))
base_model.add(layers.Dense(256,activation='relu'))
base_model.add(layers.Dense(128,activation='relu'))
base_model.add(layers.Dense(10,activation='softmax'))
```

In [80]:
```
xavier_model=models.Sequential()
xavier_model.add(layers.Flatten(input_shape=(32,32,3)))
xavier_model.add(layers.Dense(256,activation='relu',kernel_initializer=GlorotNormal
xavier_model.add(layers.Dense(128,activation='relu',kernel_initializer=GlorotNormal
xavier_model.add(layers.Dense(10,activation='softmax'))
```

In [81]:
```
kaiming_model=models.Sequential()
kaiming_model.add(layers.Flatten(input_shape=(32,32,3)))
kaiming_model.add(layers.Dense(256,activation='relu',kernel_initializer=HeNormal())
kaiming_model.add(layers.Dense(128,activation='relu',kernel_initializer=HeNormal())
kaiming_model.add(layers.Dense(10,activation='softmax'))
```

In [82]:
```
model_dropout=models.Sequential()
model_dropout.add(layers.Flatten(input_shape=(32,32,3)))
model_dropout.add(layers.Dense(256,activation='relu'))
model_dropout.add(Dropout(0.5))
model_dropout.add(layers.Dense(128,activation='relu'))
model_dropout.add(Dropout(0.5))
model_dropout.add(layers.Dense(10,activation='softmax'))
```

In [83]:
```
model_l2=models.Sequential()
model_l2.add(layers.Flatten(input_shape=(32,32,3)))
model_l2.add(layers.Dense(256,activation='relu',kernel_regularizer=regularizers.L2(
model_l2.add(layers.Dense(128,activation='relu',kernel_regularizer=regularizers.L2(
model_l2.add(layers.Dense(10,activation='softmax'))
```

In [84]:
```
model_l1=models.Sequential()
model_l1.add(layers.Flatten(input_shape=(32,32,3)))
model_l1.add(layers.Dense(256,activation='relu',kernel_regularizer=regularizers.L1(
model_l1.add(layers.Dense(128,activation='relu',kernel_regularizer=regularizers.L1(
model_l1.add(layers.Dense(10,activation='softmax'))
```

In [85]:
```
base_model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=
```

In [86]:
```
xavier_model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metric
```

In [87]:
```
kaiming_model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metri
```

In [88]:
```
model_dropout.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metri
```

In [89]:
```
model_l2.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['
```

In [90]:
```
model_l1.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['
```

In [92]:
```
base_history=base_model.fit(x_train,y_train,epochs=5,validation_data=(x_test,y_test
```

```
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.4695 - loss: 1.4741 - va
l_accuracy: 0.4624 - val_loss: 1.5102
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4774 - loss: 1.4570 - va
l_accuracy: 0.4526 - val_loss: 1.5437
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4846 - loss: 1.4388 - va
l_accuracy: 0.4715 - val_loss: 1.5005
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.4908 - loss: 1.4245 - val
_accuracy: 0.4797 - val_loss: 1.4777
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.5019 - loss: 1.4000 - va
l_accuracy: 0.4755 - val_loss: 1.4794
```

In [93]:
```
xavier_history=xavier_model.fit(x_train,y_train,epochs=5,validation_data=(x_test,y_
```

```
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.2794 - loss: 1.9981 - va
l_accuracy: 0.3920 - val_loss: 1.7017
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.3850 - loss: 1.7099 - val
_accuracy: 0.4040 - val_loss: 1.6470
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4173 - loss: 1.6235 - va
l_accuracy: 0.4030 - val_loss: 1.6889
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4391 - loss: 1.5673 - va
l_accuracy: 0.4431 - val_loss: 1.5411
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4460 - loss: 1.5375 - va
l_accuracy: 0.4572 - val_loss: 1.5315
```

In [94]:
```
kaiming_history=kaiming_model.fit(x_train,y_train,epochs=5,validation_data=(x_test,
```

```
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.2783 - loss: 2.0250 - va
l_accuracy: 0.3847 - val_loss: 1.7026
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.3884 - loss: 1.7007 - va
l_accuracy: 0.4119 - val_loss: 1.6400
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4261 - loss: 1.6086 - va
l_accuracy: 0.4243 - val_loss: 1.5777
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4450 - loss: 1.5498 - va
l_accuracy: 0.4402 - val_loss: 1.5693
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.4526 - loss: 1.5265 - va
l_accuracy: 0.4445 - val_loss: 1.5517
```

In [95]:
```
dropout_history=model_dropout.fit(x_train,y_train,epochs=5,validation_data=(x_test,
```

```
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.1245 - loss: 2.3137 - va
l_accuracy: 0.1734 - val_loss: 2.1562
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.1345 - loss: 2.2285 - va
l_accuracy: 0.1724 - val_loss: 2.1405
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.1336 - loss: 2.2248 - va
l_accuracy: 0.1640 - val_loss: 2.2004
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.1355 - loss: 2.2261 - va
l_accuracy: 0.1764 - val_loss: 2.1601
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 10s 6ms/step - accuracy: 0.1402 - loss: 2.2234 - va
l_accuracy: 0.1768 - val_loss: 2.1679
```

In [96]:
```
l2_history=model_l2.fit(x_train,y_train,epochs=5,validation_data=(x_test,y_test))
```

```
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.2603 - loss: 3.1363 - va
l_accuracy: 0.2908 - val_loss: 2.0203
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.3262 - loss: 1.9778 - va
l_accuracy: 0.3601 - val_loss: 1.8986
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.3505 - loss: 1.9035 - va
l_accuracy: 0.3720 - val_loss: 1.8491
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.3532 - loss: 1.8872 - va
l_accuracy: 0.3550 - val_loss: 1.8755
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.3657 - loss: 1.8585 - va
l_accuracy: 0.3672 - val_loss: 1.8567
```

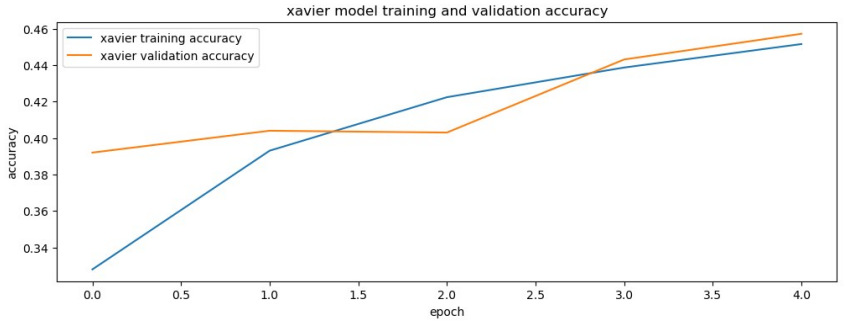
In [109…
```
l1_history=model_l1.fit(x_train,y_train,epochs=5,validation_data=(x_test,y_test))
```

```
Epoch 1/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 13s 8ms/step - accuracy: 0.1008 - loss: 3.2002 - va
l_accuracy: 0.1000 - val_loss: 3.2004
Epoch 2/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.0979 - loss: 3.2002 - va
l_accuracy: 0.1000 - val_loss: 3.2002
Epoch 3/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.1008 - loss: 3.2001 - va
l_accuracy: 0.1000 - val_loss: 3.2003
Epoch 4/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.1018 - loss: 3.2003 - va
l_accuracy: 0.1000 - val_loss: 3.2001
Epoch 5/5
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.0996 - loss: 3.2003 - va
l_accuracy: 0.1000 - val_loss: 3.1983
```

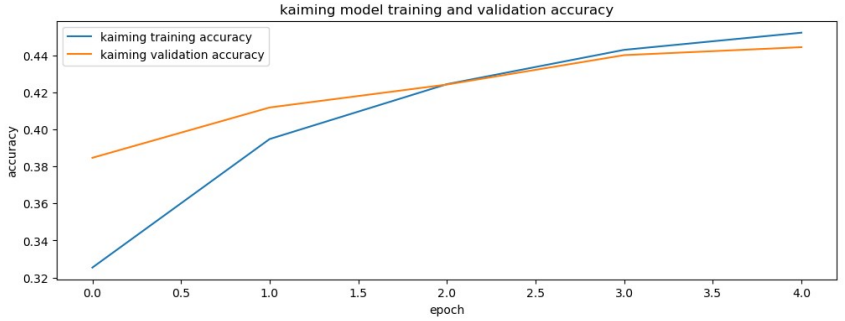
In [110…
```
plt.figure(figsize=(12,4))
plt.plot(base_history.history['accuracy'],label='base training accuracy')
plt.plot(base_history.history['val_accuracy'],label='base validation accuracy')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend()
plt.title('base model training and validation accuracy')
plt.show()
```



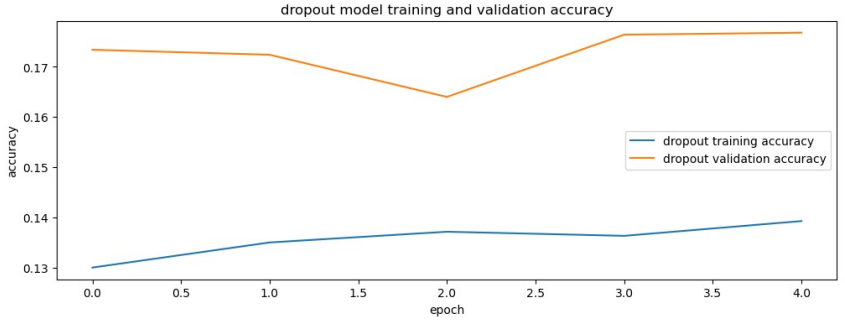
In [111…
```
plt.figure(figsize=(12,4))
plt.plot(xavier_history.history['accuracy'],label='xavier training accuracy')
plt.plot(xavier_history.history['val_accuracy'],label='xavier validation accuracy')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend()
plt.title('xavier model training and validation accuracy')
plt.show()
```



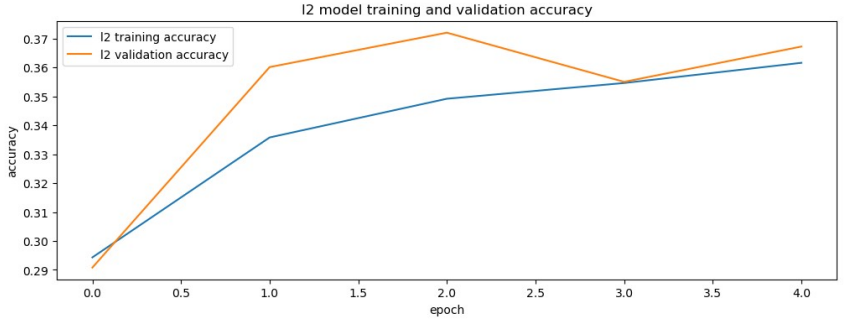
In [112…
```
plt.figure(figsize=(12,4))
plt.plot(kaiming_history.history['accuracy'],label='kaiming training accuracy')
plt.plot(kaiming_history.history['val_accuracy'],label='kaiming validation accuracy')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend()
plt.title('kaiming model training and validation accuracy')
plt.show()
```



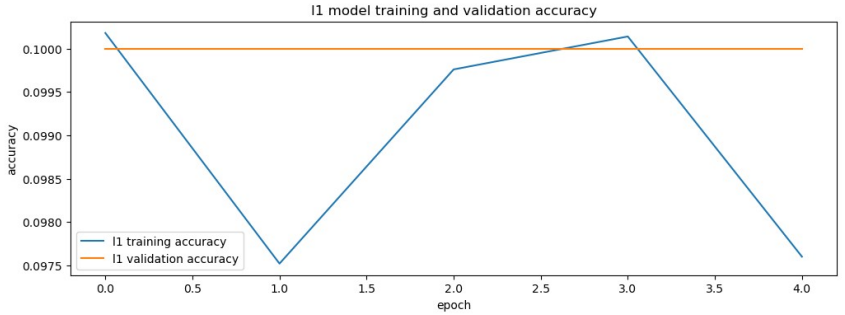
In [113…
```
plt.figure(figsize=(12,4))
plt.plot(dropout_history.history['accuracy'],label='dropout training accuracy')
plt.plot(dropout_history.history['val_accuracy'],label='dropout validation accuracy')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend()
plt.title('dropout model training and validation accuracy')
plt.show()
```



In [114…
```
plt.figure(figsize=(12,4))
plt.plot(l2_history.history['accuracy'],label='l2 training accuracy')
plt.plot(l2_history.history['val_accuracy'],label='l2 validation accuracy')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend()
plt.title('l2 model training and validation accuracy')
plt.show()
```



In [115…
```
plt.figure(figsize=(12,4))
plt.plot(l1_history.history['accuracy'],label='l1 training accuracy')
plt.plot(l1_history.history['val_accuracy'],label='l1 validation accuracy')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend()
plt.title('l1 model training and validation accuracy')
plt.show()
```



In [116…
```
base_pred=base_model.predict(x_test)
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

In [117…
```
xavier_pred=xavier_model.predict(x_test)
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

In [118…
```
kaiming_pred=kaiming_model.predict(x_test)
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

In [119…
```
dropout_pred=model_dropout.predict(x_test)
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

In [120…
```
l2_pred=model_l2.predict(x_test)
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

In [121…
```
num_images_to_display=10
class_names=['airplane','automobile','bird','cat','deer','dog','frog','house','ship
plt.figure(figsize=(20,9))
for i in range(num_images_to_display):
    plt.subplot(2,5,i+1)
    plt.imshow(x_test[i])
    predicted_label=np.argmax(base_pred[i])
    true_label=y_test[1][0]
    plt.title(f'pred:{class_names[predicted_label]}\n true:{class_names[true_label]
```



In [ ]: